# Unconstrained optimization

INTRODUCTION TO OPTIMIZATION IN PYTHON



**Jasmin Ludolf**
Content Developer

datacamp

# What is unconstrained optimization?

- Finding the maxima or minima of a function that does not have any constraints on the input variables

- `SciPy`



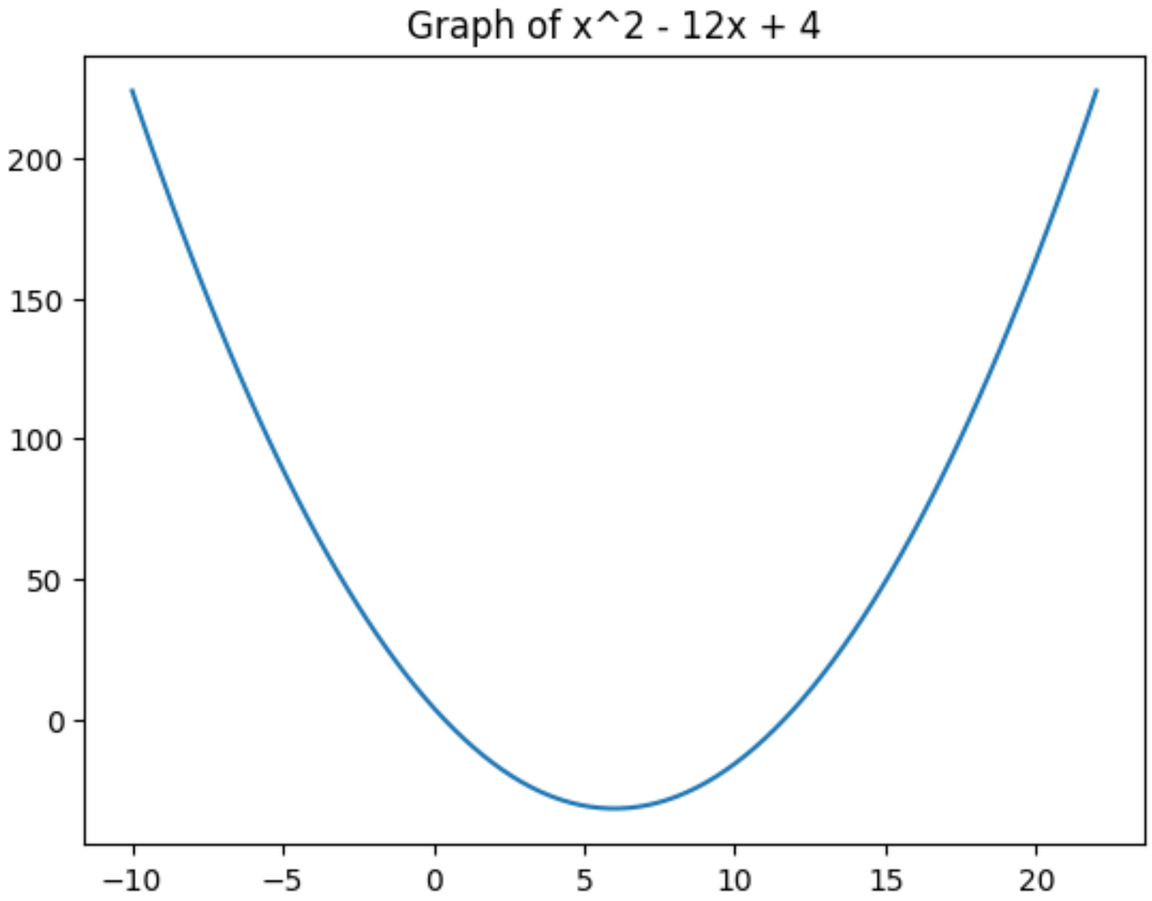
# Univariate unconstrained optimization

```
from scipy.optimize import minimize_scalar

def objective_function(x):
  return x**2 - 12*x + 4

result = minimize_scalar(objective_function)
print(result)
```

```
     fun: -32.0
 message: 'Optimization terminated successfully; The
 returned value satisfies the termination criteria
 (using xtol = 1.48e-08 )'
    nfev: 10
     nit: 4
 success: True
       x: 6.000000000000001
```



# Univariate unconstrained optimization result

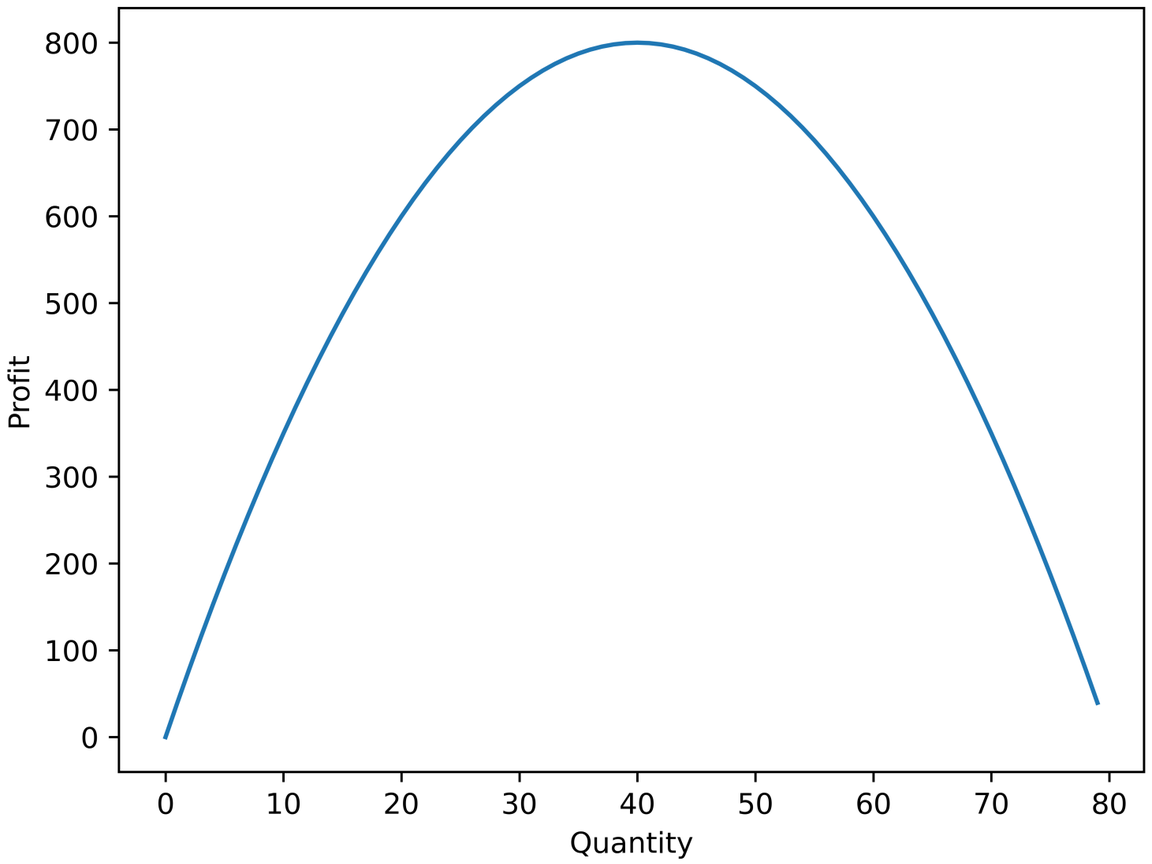
```
     fun: -32.0
 message: 'Optimization terminated successfully; The
 returned value satisfies the termination criteria
 (using xtol = 1.48e-08 )'
    nfev: 10
     nit: 4
 success: True
       x: 6.000000000000001
```

```
result.x
```

```
6.000000000000001
```

- `fun` : value of the function at the minimum
- `message` : status
- `nfev` : number of times the algorithm evaluated the function
- `nit` : number of iterations it took to reach the solution
- `success` : boolean on whether an optimum was found
- `x` : the optimal value

# Finding the maxima




```
def objective_function(x):
  return 40 * q - 0.5 * q**2
```

```
-1*(40 * q - 0.5 * q**2)
-40 * q + 0.5 * q**2
```

```
def negated_function(x):
  return -40 * q + 0.5 * q**2
result = minimize_scalar(negated_function)
print(f"The maximum is {result.x:.2f} in two
      decimals")
```

```
The maximum is 40.00 in two decimals
```

# Multivariate unconstrained optimization

```
from scipy.optimize import minimize

def objective_function(a):
  return (a[0] - 2)**2 + (a[1] - 3)**2 + 1.5

x0 = [1, 2]

result = minimize(objective_function, x0)
```

- `a[0]` represents `x` and `a[1]` represents `y`

# Multivariate unconstrained optimization result

```
print(result)
print(f"minimum is (x, y) = ({result.x[0]:.2f}, {result.x[1]:.2f}) in two decimals.")
```

```
      fun: 1.5000000000000002
 hess_inv: array([[ 0.75, -0.25],
       [-0.25,  0.75]])
      jac: array([0., 0.])
  message: 'Optimization terminated successfully.'
     nfev: 9
      nit: 2
     njev: 3
   status: 0
  success: True
        x: array([1.99999999, 2.99999999])
minimum is (x, y) = (2.00, 3.00) in two decimals.
```

# Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

# Bound-constrained optimization

INTRODUCTION TO OPTIMIZATION IN PYTHON



**Jasmin Ludolf**
Content Developer

datacamp

# Bound constrained optimization

- Default solver: Broyden-Fletcher-Goldfarb-Shanno (BFGS)
- The constraint is a range
- Produce between 50 and 100 units per day

```
from scipy.optimize import Bounds

bounds = Bounds(50, 100)
```

# L-BFGS-B

```
from scipy.optimize import minimize, Bounds

def objective_function(b):
  return (2*b[0] - 1.5*b[1])
bounds = Bounds([5, 10], [25, 30])
x0 = [10, 5]
result = minimize(objective_function, x0, method='L-BFGS-B', bounds=bounds)

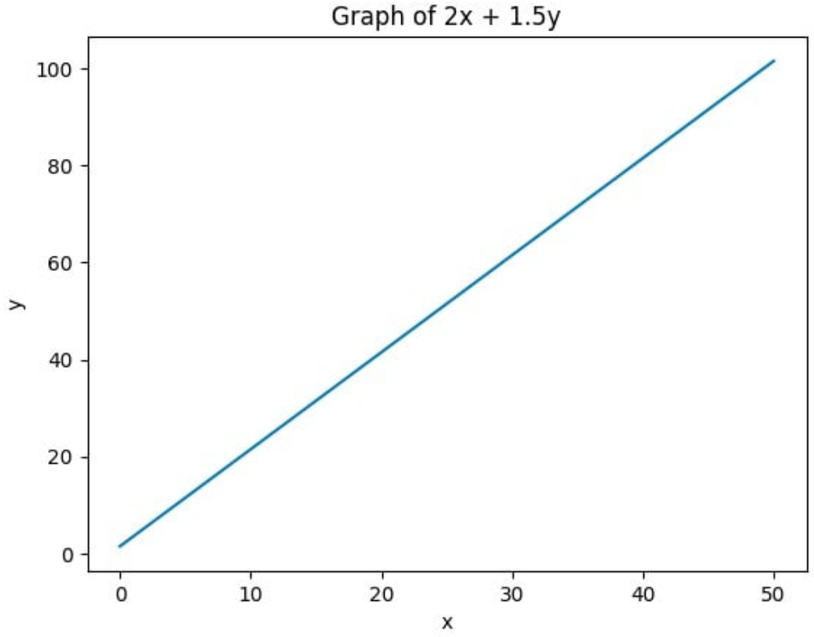
print(result.x)
```

```
[ 5. 30.]
```

# Linear constrained optimization

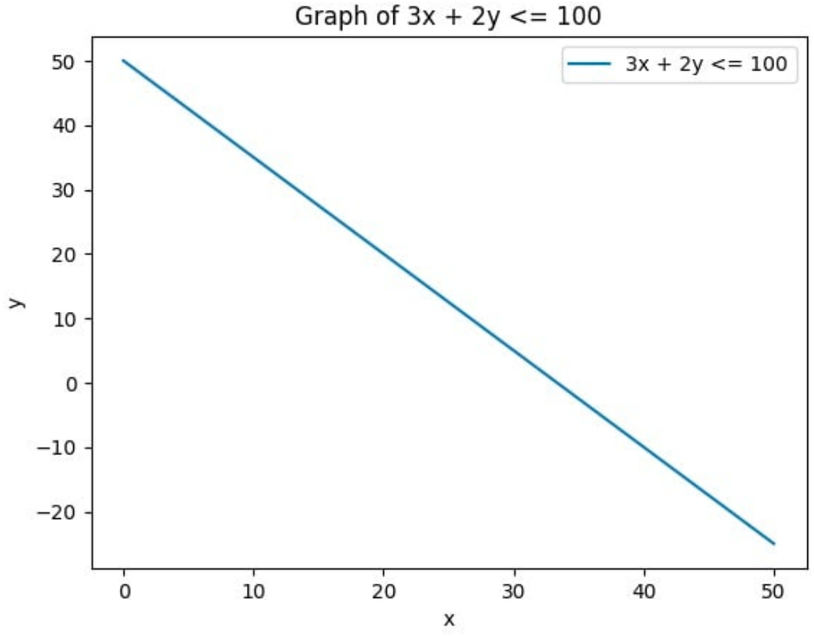
## Objective function OR

- Linear
- $p = 2x + 1.5y$



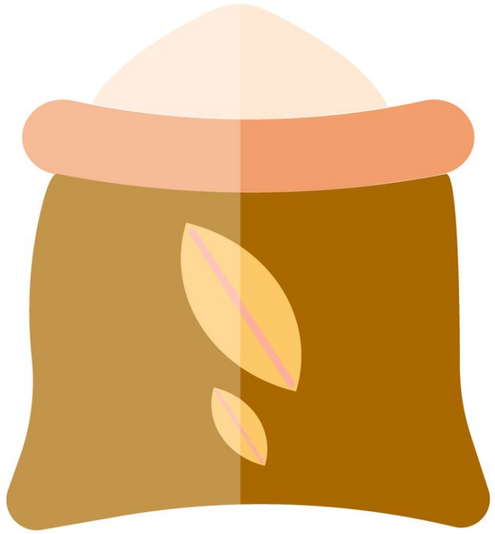

## Constraint

- Linear
- $3x + 2y <= 100$



# Hard inequality

- A condition
- No more biscuits once flour is used!



# Solve a linear constrained problem

```
from scipy.optimize import minimize

def objective_function(b):
  return (b[0]) ** 2 + (b[1]) ** 0.5

def constraint_function(x):
    return 2*x[0] + 3*x[1] - 6

constraint = {'type': 'ineq',
              'fun': constraint_function}
x0 = [20, 20]
result = minimize(objective_function, x0,
         constraints=[constraint])

print(result)
```

```
     fun: 1.4000405652399073
     jac: array([0.24057153, 0.36086182])
 message: 'Optimization terminated successfully'
    nfev: 25
     nit: 8
    njev: 8
  status: 0
 success: True
       x: array([0.12028576, 1.9198095 ])
```

- Types of contraints:
  - `'eq'` : equality
  - `'ineq'` : inequality

# Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON

# Linear programming

## INTRODUCTION TO OPTIMIZATION IN PYTHON



**Jasmin Ludolf**
Content Developer

datacamp

# Linear programming

- Objective functions and constraints have a linear relationship
- Opposed to linear constrained: mix of linear or non linear elements

# Product mix

| Type | Count | Bottles (h/box) | Cups (h/box) |
|---|---|---|---|
| $M_A$ | 4 | 1.5 | 1.3 |
| $M_B$ | 3 | 0.8 | 2.1 |

- Each machine operates for 30 hours per week

- Demand: bottles for drinks, cups for yogurt

# Maximize profit

- **Profit per box**

| B ($/box) | C ($/box) |
|---|---|
| 480 | 510 |

- Objective function: $480B + 510C$

- Constraints
  - Machine A: $1.5B + 1.3C \leq 4 \times 30$
  - Machine B: $0.8B + 2.1C \leq 3 \times 30$
  - Non-negativity constraint: $B, C \geq 0$

# Using PuLP

```
from pulp import *

model = LpProblem('MaxProfit', LpMaximize)

B = LpVariable('B', lowBound=0)
C = LpVariable('C', lowBound=0)

model += 480*B + 510*C
model += 1.5*B + 1.3*C - 120, "M_A"
model += 0.8*B + 2.1*C - 90, "M_B"
```

- `LpMaximize` or `LpMinimize`

# PuLP model output

```
print(model)
```

```
ProductMix:
MAXIMIZE
480*B + 510*C + 0
SUBJECT TO
M_A: 1.5 B + 1.3 C <= 120

M_B: 0.8 B + 2.1 C <= 90

VARIABLES
B Continuous
C Continuous
```

# Solving with PuLP

```
status = model.solve()
print(status)
```

```
Welcome to the CBC MILP Solver
...
1
```

```
print(f"Profit = {value(model.objective):.2f}")
print(f"Tons of bottles = {B.varValue:.2f}, tons of cups = {C.varValue:.2f}")
```

```
Profit = 40137.44
Boxes of bottles = 63.98, boxes of cups = 18.48
```

# Handling multiple variables and constraints

```
variables = LpVariable.dicts("Product", ['B', 'C'], lowBound=0)
```

OR

```
variables = LpVariable.dicts("Product", range(2), 0)
```

```
A = LpVariable.matrix('A', (range(2), range(2)), 0)
```

```
box_profit = {'B': 480, 'C': 510}
model += lpSum([box_profit[i] * variables[i] for i in ['B', 'C']])

model += 1.5*variables['B'] + 1.3*variables['C'] <= 120, "M_A"
model += 0.8*variables['B'] + 2.1*variables['C'] <= 90, "M_B"
```

# Let's practice!

INTRODUCTION TO OPTIMIZATION IN PYTHON